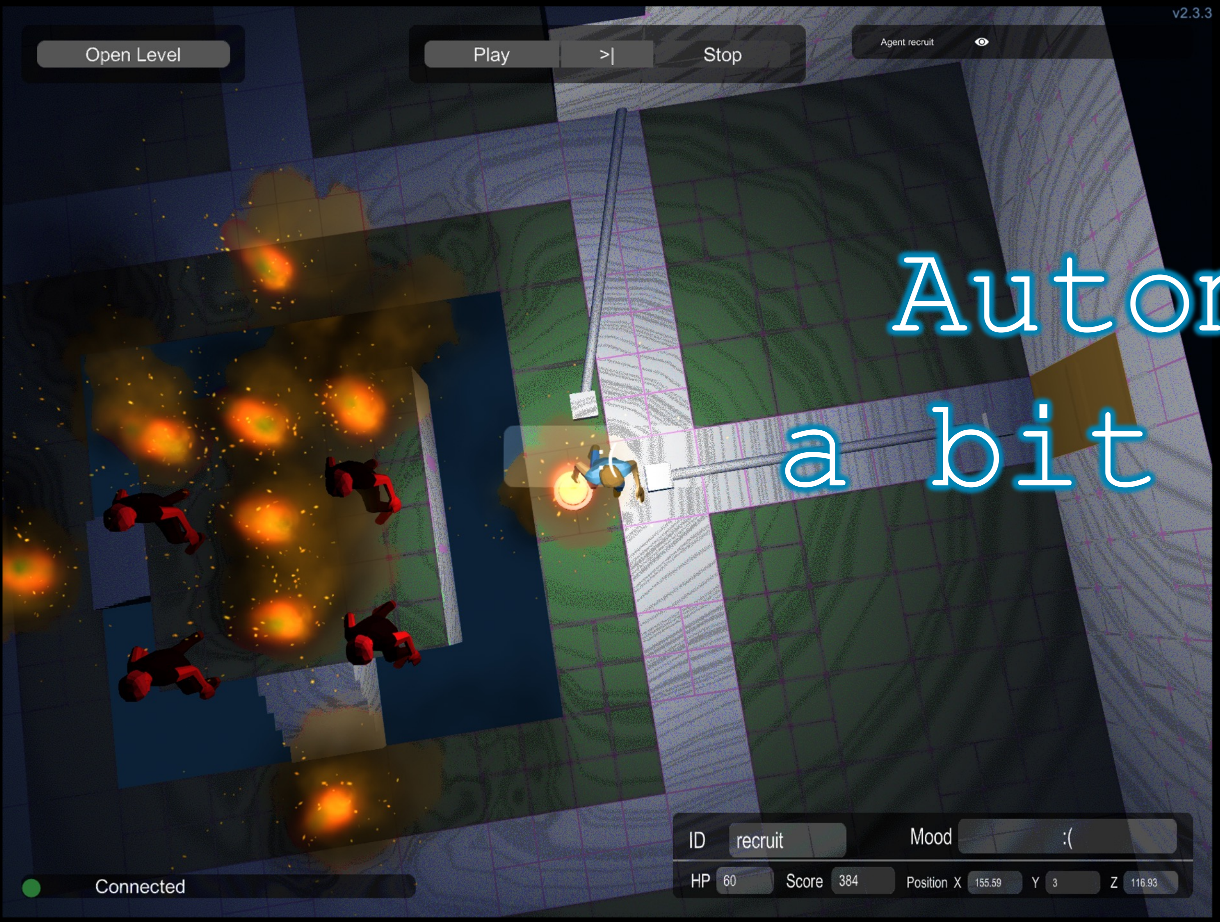


Automated Testing
a bit about research

Automated testing

In the industry it means that you are able to automatically (and not manuallu) execute your tests.

Essentiall for modern agile approach.

In research means generating tests:

- generating the test inputs and interactions
- in the absence of post-cond: generating the oracles/assertions

Generating tests

- Goal: maximize coverage
- In the absence of post-cond: generate regression assertions, you need to minimimize generated test-cases as manual inspection is needed
- Approaches:
  - random, fuzzing
  - search based testing
  - symbolic execution

Search based testing (SBT)

Randomly generate or seed initial population P
Repeat
    Evaluate $\boxed{\text{fitness}}$ of each individual in P
    Select parents from P according to selection mechanism
    Recombine parents to form new offspring
    Construct new population P' from parents and offspring
    Mutate P'
    $P \leftarrow P'$
Until Stopping Condition Reached

Figure 3. High-level description of a genetic algorithm.

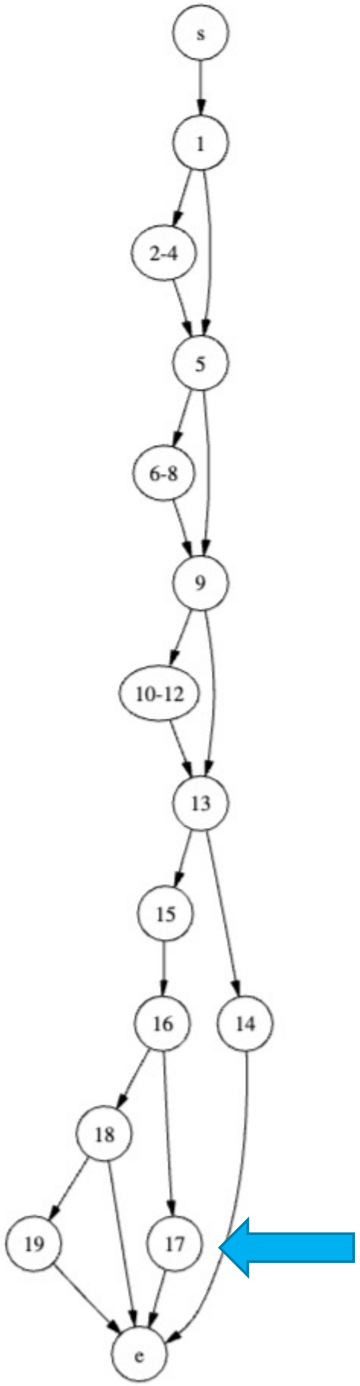
Search-based software test data generation: a survey, McMinn, STVR 2004

Fitness function

- Fitness value towards reaching node 17 could be a + 0.1v where a is the "approach level" to node-17, v is the branch-distance in the condition in line-16 towards taking the edge to 17, e.g. |a-b| + |b-c|.

- Fitness values can be measured, but unfortunately they do not tell us how the solve/minimize their values (reaching zero)

```
CFG
Node
s       int tri_type(int a, int b, int c)
        {
            int type;

1           if (a > b)
2-4         {   int t = a; a = b; b = t;    }

5           if (a > c)
6-8         {   int t = a; a = c; c = t;    }

9           if (b > c)
10-12       {   int t = b; b = c; c = t;    }

13          if (a + b <= c)
            {
14              type = NOT_A_TRIANGLE;
            }
            else
            {
15              type = SCALENE;
16              if (a == b && b == c)
                {
17                  type = EQUILATERAL;
                }
18              else if (a == b || b == c)
                {
19                  type = ISOSCELES;
                }
            }

e           return type;
        }
```

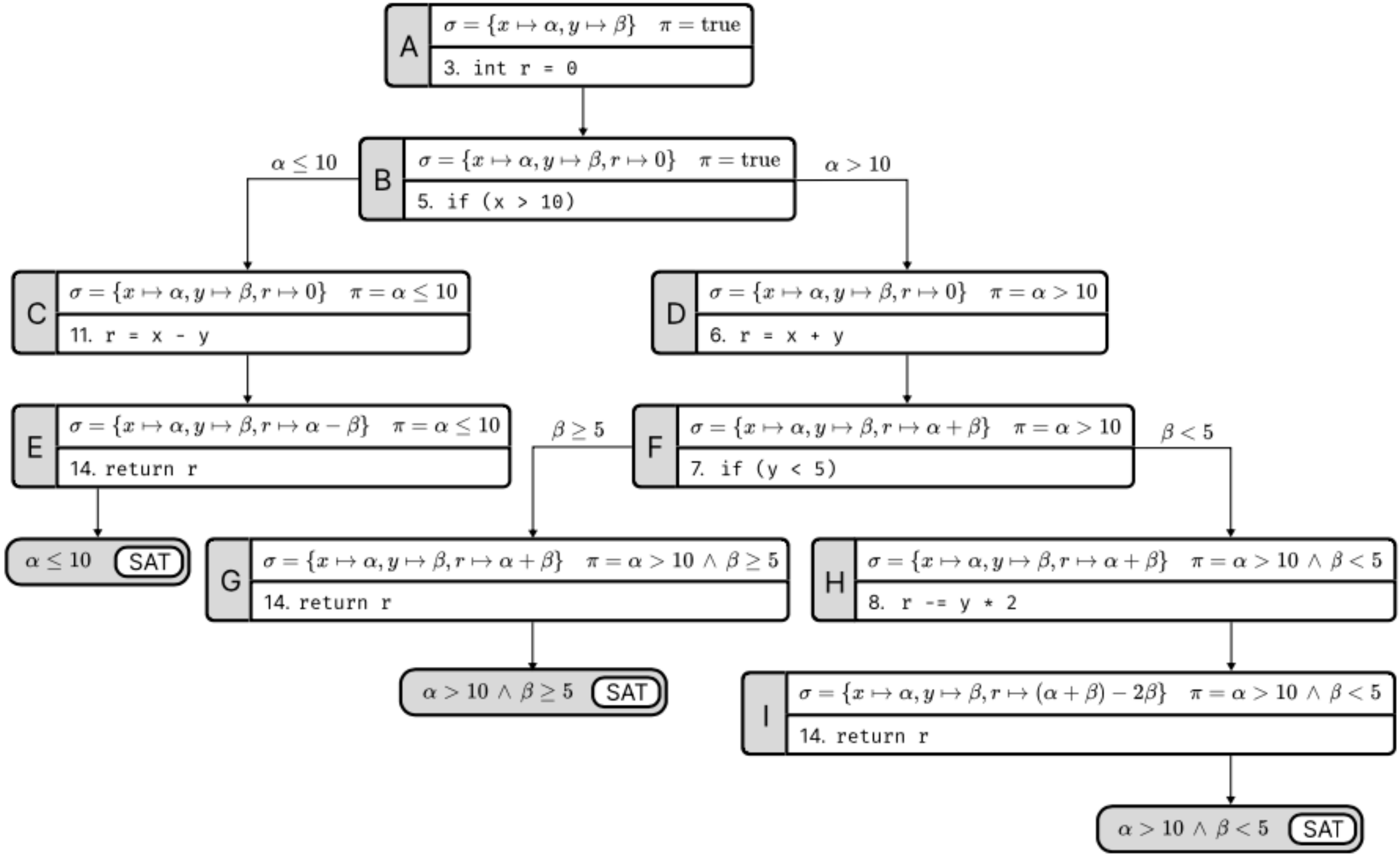


SBT tools

- SBT is considered to be the most successful automated testing approach, so far.
- Tools: Evosuite (Java), Pynguin (Python) for unit testing.

Symbolic approach

```
public class SimpleExample {
    public int foobar(int x, int
        int r = 0;

        if (x > 10) {
            r = x + y;
            if (y < 5) {
                r -= y * 2;
            }
        } else {
            r = x - y;
        }

        return r;
    }
}
```



Evaluating Search Strategies in Dynamic Symbolic Execution for Java Test Generation, Kroon, master thesis UU, 2025.

MAZE

- Leveraging semantic information.
- Challanges: dealing with path explosion, dealing with program calls

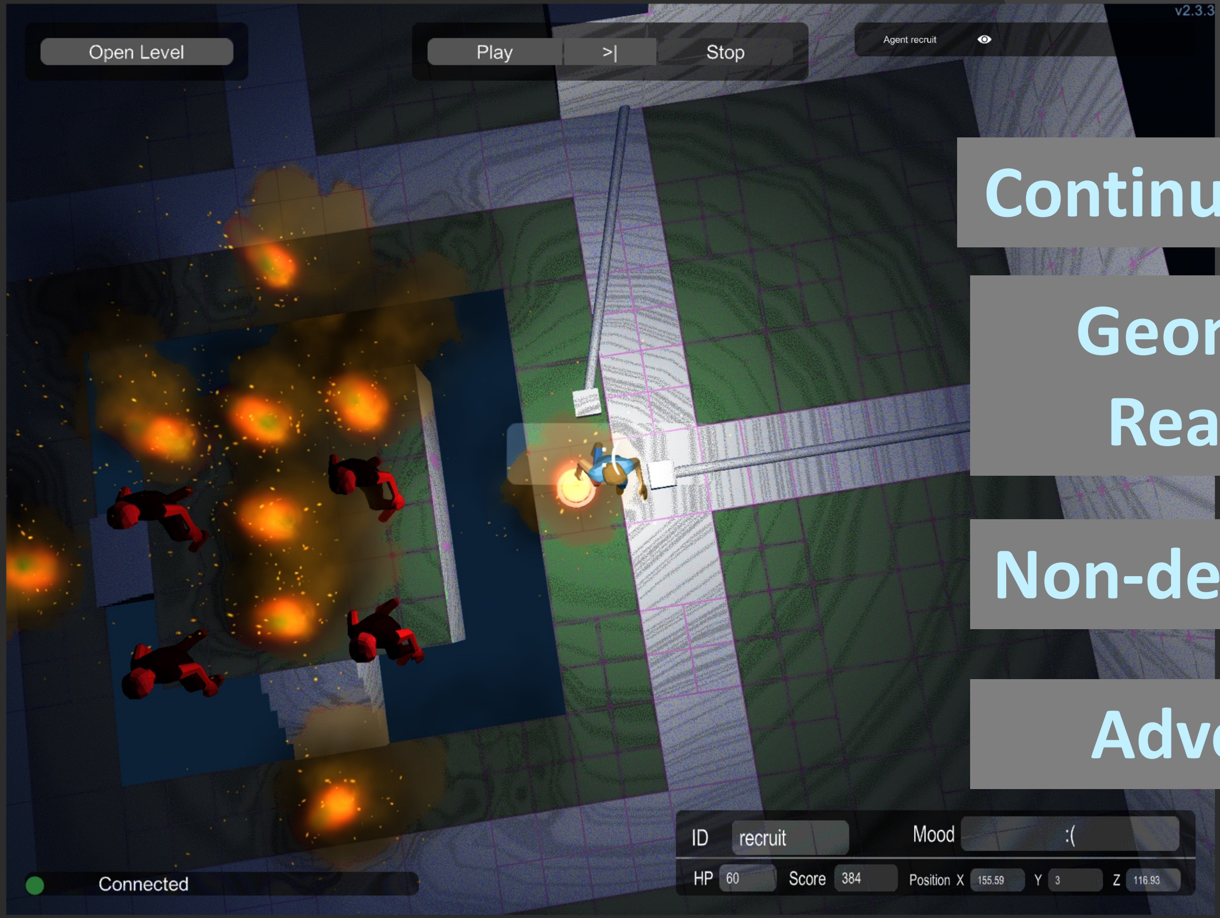
Symbolic-based tools

Automated testing at the system level

- Well... the state space is HUGE.
- Critical states could be very deep in the state space.
- Very very challenging for fully automated approaches like SBT or Symbolic
- Here at UU we have research in automated game testing

Challanges in Automated Game Testing

~~Model-based testing~~
~~Reinforcement learning~~
~~Search-based testing~~
~~Combinatorial testing~~



Continuous Space

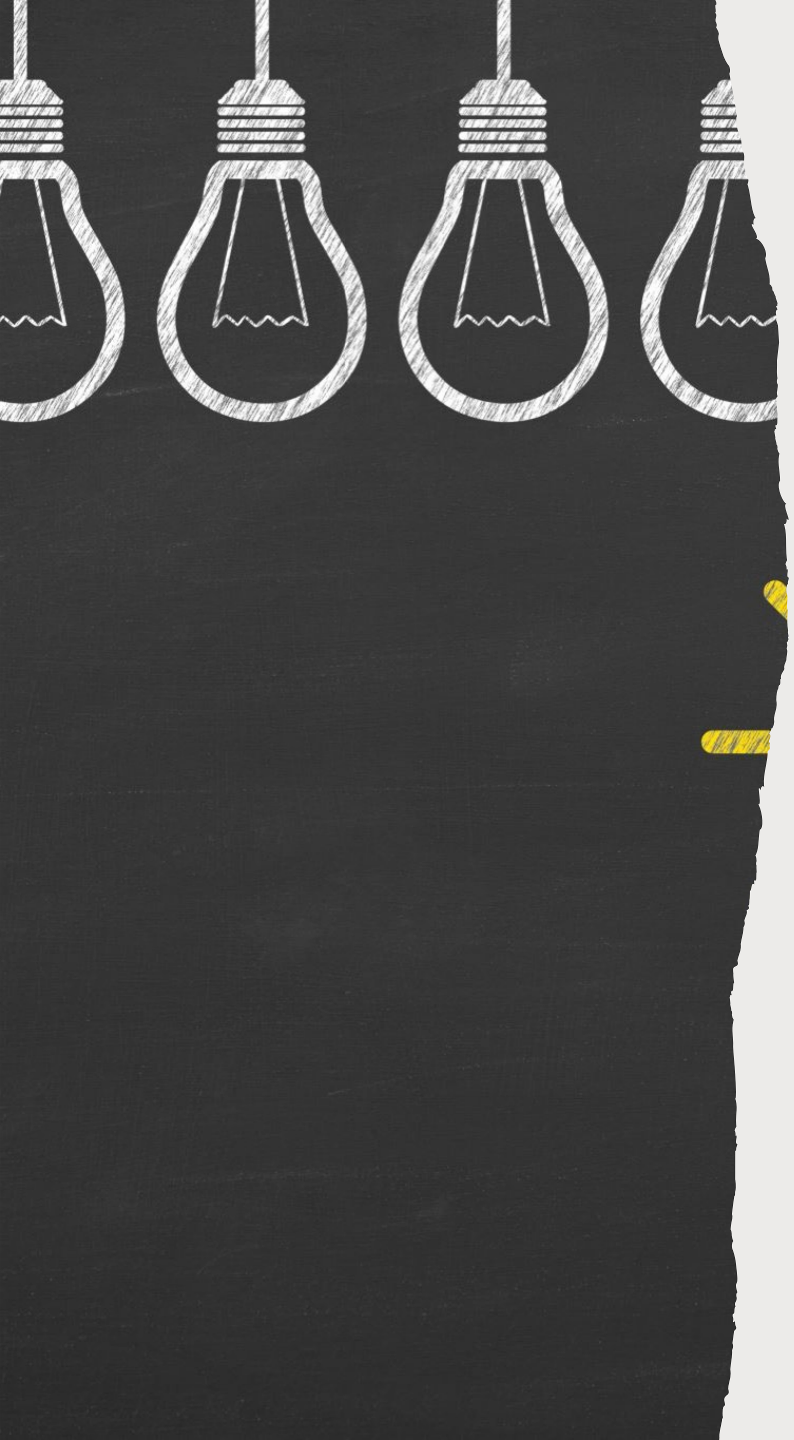
Geometrical Reasoning

Non-determinism

Adversarial

Typical execution in a game

- It runs in cycles (frame-updates). At each cycle:
  - calculate the effect of user inputs
  - calculate the effect of autonomous in-game entities
  - update the game state
- Action: user command for what to do in a single frame-update, e.g. “move-forward 1 unit”



An agent-based solution

- Agent-programming: **reactive**.
- Intelligent-agent paradigm: goals, tactics, emotion.
- Multi-agent

Agent tactical programming

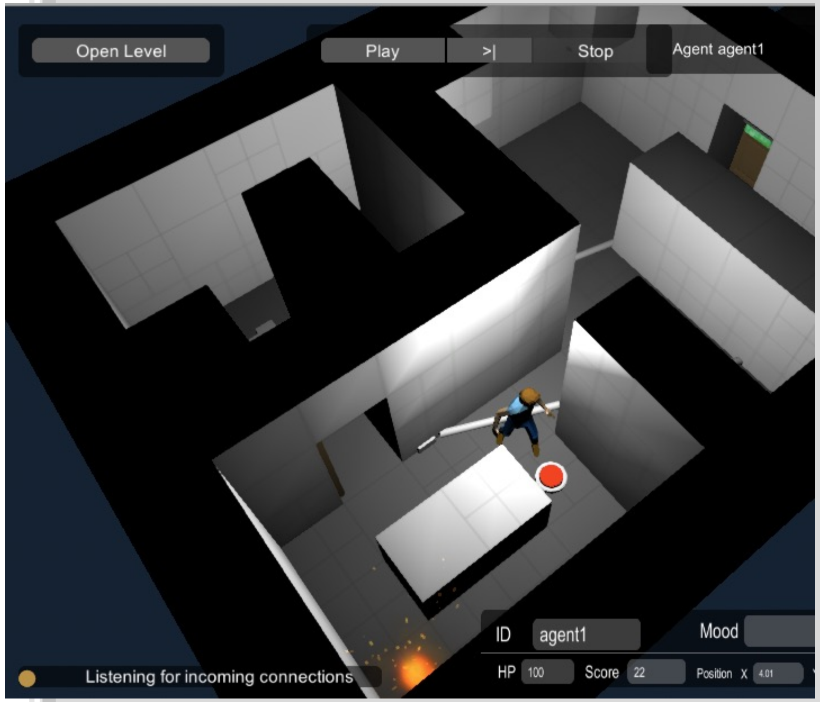
```
var tactic1 = ANYof(
   action(). do1(B → B.env().moveUp()).  on(g1)
   action(). do1(B → B.env().moveDown()).   on(g2)
   ....
)
```

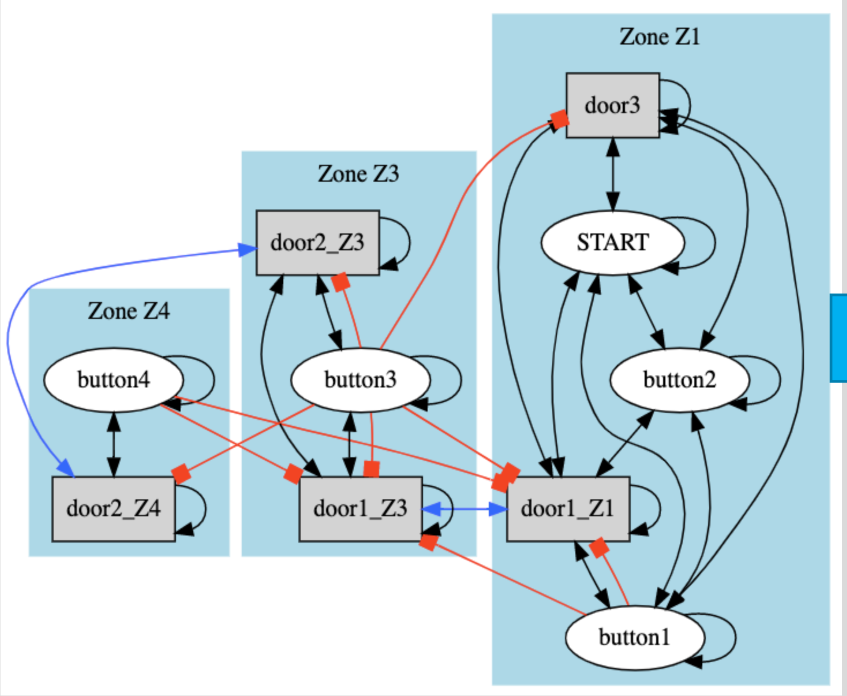
$$
\begin{array}{lll}
\textbf{var} & \mathit{tactic}_2 & = \ \mathsf{FIRSTof}(\\
& \multicolumn{2}{l}{\mathsf{action}().\mathsf{do1}(B \longrightarrow B.\mathit{env}().\mathit{useHealKit}()).\mathsf{on}(g)\ ,}\\
& \multicolumn{2}{l}{\mathit{navigateTo}(k)\ ,}\\
& \multicolumn{2}{l}{\mathit{explore}())}
\end{array}
$$

Goal programming

$$
\begin{array}{lll}
\mathrm{SEQ}(& \text{"}k\text{ is found"} & .\mathrm{withTactic}(T_1(k)),\\
& \text{"}k\text{ is picked up"} & .\mathrm{withTactic}(T_2(k)),\\
& \text{"}d\text{ is found"} & .\mathrm{withTactic}(T_1(d)),\\
& \text{"}k\text{ is used on }d\text{"} & .\mathrm{withTactic}(T_3(k,d)))
\end{array}
$$

Future direction: automated model inference followed by test generation





generate

- Abstract test-cases in the form goal-structures.
- Give them to agent for execution